

Universidade Federal do Rio Grande do Norte
Estruturas Algbricas DIM0167 - 2026.2

Especificação Algébrica de Sistema de Biblioteca em CafeOBJ

Amauri Lacerda de Brito Junior
Kael Amir Pina Paraguassu Silva
Gabriel Victor da Silva
Pedro Lucas Medeiros do Nascimento
Samuel Morais da Rocha e Silva

Natal, 2 de Novembro de 2026.

1 Introdução

2 Visão Geral do Sistema e Regras de Negócio

O sistema de biblioteca é composto por três coleções principais:

- Acervo: multiconjunto de livros pertencentes à biblioteca.
- Usuários: multiconjunto de usuários registrados.
- Empréstimos: multiconjunto de pares (livro, usuário) que representam empréstimos ativos.

Cada livro possui um identificador único (LivroId), uma obra associada (ObraId), título, autor e ano. Cada usuário possui identificador único (UsuarioId), nome e e-mail. As regras de negócio definidas são:

1. Um livro não pode ser emprestado se já estiver emprestado.
2. A devolução de um livro deve torná-lo novamente disponível.
3. Um usuário só pode devolver um livro que esteja emprestado a ele.
4. A remoção de um livro só é permitida quando ele estiver disponível.
5. A identificação de cada livro deve ser única.
6. A quantidade de livros disponíveis mais a quantidade de livros emprestados deve ser igual à quantidade total de livros do acervo.
7. Essas regras são garantidas por guardas nas transições de estado e por predicados de consistência global (valido? e emprestimosValidos?).

Para garantir tais regras, foram declaradas guardas nas transições de estados predicados de consistência global; como `valido?` e `emprestimosValidos?`

3 Assinatura

3.1 *Sorts* e *Subsorts*

A especificação declara os seguintes *sorts* e *subsorts*:

- `LivroId`, `UsuarioId`, `ObraId`: identificadores atômicos.
- `Livro`, `Usuario`: entidades.
- `Acervo`, `ConjuntoUsuarios`, `ConjuntoEmprestimos`: coleções monoidais.
- `Emprestimo`: relação livro–usuário.
- `Biblioteca`: estado global visível.
- `LivroOuErro`: *supersort* que inclui Livro e os construtores de erro `livroInexistente` e `livroDuplicado`.
- Subsorts: Livro $<$ LivroOuErro, Livro $<$ Acervo, Usuario $<$ ConjuntoUsuarios, Emprestimo $<$ ConjuntoEmprestimos.

3.2 Operadores e Construtores

3.3 Variáveis

4 Estruturas de Dados

5 Equações

5.1 Acervo

5.2 Empréstimos e devoluções

5.3 Transições globais

5.4 Invariantes globais

6 Tratamento de estados inválidos

6.1 Busca por livro inexistente

6.2 Busca por livro com ID duplicado

6.3 Remoção de entidade inexistente

6.4 Adição de elemento inválido ou com ID inexistente

6.5 Devolução de livro não emprestado

6.6 Estados inválidos em nível global

7 Termos de teste

8 Análise da especificação

8.1 Justificativa dos *sorts* e operadores

Conforme os requisitos do trabalho, foram desenvolvidos os termos de teste agregados na Tabela 1, cobrindo as operações básicas e compostas, casos inválidos e aplicação das regras de negócio.

Os *sorts* `LivroId, UsuarioId e ObraId` foram separados para garantir unicidade e evitar confusão entre identificadores de naturezas distintas. `Livro` e `Usuario` são entidades com construtores e projeções, permitindo validação (`valido?`) e consultas. As coleções foram modeladas como monóides comutativos, já que a ordem dos elementos não é relevante para as consultas, e a composição é naturalmente associativa e comutativa. O *sort* `Biblioteca` consolida o estado global e permite transições atômicas e guardadas.

Tabela 1: Termos de teste e resultados esperados.

#	Termo de teste	Resultado esperado
1	`qtdAcervo(adicionarLivro(livro(l1, o1, "T1", "A1", 2020), vazioAcervo))`	`1`
2	`pertenceAcervo?(l1, adicionarLivro(livro(l1, o1, "T1", "A1", 2020), vazioAcervo))`	`true`
3	`buscarLivro(l2, adicionarLivro(livro(l1, o1, "T1", "A1", 2020), vazioAcervo))`	`livroInexistente`
4	`qtdUsuarios(adicionarUsuario(usuario(u1, "Ana", "ana@x"), vazioUsuarios))`	`1`
5	`emprestado?(l1, emprestar(l1, u1))`	`true`
6	`temLivro?(u1, l1, emprestar(l1, u1))`	`true`
7	`disponivel?(l1, emprestarLivro(l1, u1, B))`	`false` (após empréstimo válido)
8	`qtdEmprestimos(emprestarLivro(l1, u1, B))`	`1`
9	`emprestarLivro(l1, u2, B)` com `u2` não registrado	`B` (inalterada)
10	`removerLivro(l1, emprestarLivro(l1, u1, B))`	`B` (livro emprestado não é removido)
11	`devolverLivro(l1, u2, B)` com empréstimo de `u1`	`B` (inalterada)
12	`qtdDisponiveis(B)` + `qtdEmprestimos(B)`	`qtdAcervo(B)` (invariante)

8.2 Representatividade do modelo

A especificação representa as principais características do sistema: acervo, usuários, empréstimos, disponibilidade, pertinência, contagens e regras de negócio. As transições globais refletem o ciclo de vida de um livro (registro, empréstimo, devolução, remoção) e de um usuário (registro, remoção condicionada). As invariantes `valido?` e `emprestimosValidos?` capturam a consistência global.

8.3 Características não representadas

Algumas características do mundo real não foram modeladas, por não serem exigidas pelo escopo mínimo:

1. datas de empréstimo e devolução, prazos e multas;
2. limites de quantidade de empréstimos por usuário;
3. reservas e filas de espera;
4. histórico de empréstimos encerrados;
5. informações bibliográficas mais específicas (editora, edição etc).

Essas extensões poderiam ser incorporadas com novos *sorts* e equações, sem alterar a estrutura das coleções.

8.4 Propriedades matemáticas expressas

A especificação expressa:

- Monóides comutativos para acervo, usuários e empréstimos, com identidade e associatividade;
- Idempotência de filtros e remoções quando o elemento não existe;
- Conservação de cardinalidade: `qtdDisponiveis(B) + qtdEmprestimos(B) = qtdAcervo(B);`
- Unicidade de identificadores via `idsUnicos?`;
- Validade global via `valido?`.

8.5 Representação e mitigação de operações inválidas

As operações inválidas são tratadas de maneiras distintas nas diferentes partes do sistema:

- Consultas ao acervo: No módulo `acervo.mod`, é utilizado o *supersort* `LivroOuErro` que inclui dois construtores de erro: `livroInexistente`, retornado pela busca a um livro cujo ID não existe no acervo; e `livroDuplicado`), se o ID consultado ocorre múltiplas vezes; Essa decisão permite distinguir a ausência, presença ou quebra da unicidade de um livro a partir de um tipo bem definido.
- Guardas nos modificadores de coleção: Os modificadores são guardados por equações que, mediante falha na pré-condição, retornam a coleção original. Desta forma, todas as operações são totais a nível de coleção. Este comportamento foi implementado nas operações.
  - Adição de livros e usuários: Insere apenas se o objeto - livro ou usuário - é válido e se seu ID já não pertence à coleção; se falhar nestas condições, devolve a coleção.
  - Remoção de livros e usuários: Se o ID do objeto não existe, retorna o estado atual da coleção. Essa abordagem torna a operação idempotente.
  - Devolução de livros: preserva a coleção de empréstimos empréstimo não está presente.
- Guardas nas transições globais: No módulo de `Biblioteca`, as transições globais são verificadas por meio de observadores, `disponivel?, usuarioRegistrado? e temLivro?`; se não são satisfeitos, a bibioteca original é retornada. Tal como nos modificadores de coleção, esta abordagem faz o sistema total e seguro, pois a partir de um estado válido, nenhuma transação levaria a um inválido.

A suíte de testes verifica que, após qualquer tentativa inválida, o estado permanece inalterado e os invariantes continuam válidos.